

Лабораторная работа № 4

Шифрование с открытым ключом. Алгоритм RSA

Вариант №15

Ф. И. О. студента: Рау Алексей Евгеньевич

Группа: ФИТ-231

Проверил: Белим С.Ю.

Дата: 11.11.2025

Основные сведения

Формула шифрования:

$$C = M^e \bmod n$$

Формула расшифрования:

$$M = C^d \bmod n$$

Формула, связывающая открытый ключ и закрытый ключ:

$$ed = 1 \bmod (p - 1)(q - 1)$$

Результаты

Параметр $n = 50041$

1) Открытый ключ шифрования $e = 1093$

Закрытый ключ шифрования $d = 40093$

2) Открытый ключ шифрования $e = 28727$

Закрытый ключ шифрования $d = 29591$

3) Открытый ключ шифрования $e = 48245$

Закрытый ключ шифрования $d = 3773$

Открытый текст: проверка работы

Блоки открытого текста:

$$M_1 = 2526, M_2 = 2412, M_3 = 1526, M_4 = 2010, M_5 = 9926,$$

$$M_6 = 1011, M_7 = 2428, M_8 = 37$$

Шифр-текст: 24921, 12353, 18315, 35091, 44846, 6066, 17625, 44384

Код программы

```
import math
import random
import os
import tkinter as tk
from tkinter import filedialog

RUSSIAN_ALPHABET = {
    'А': 10, 'Б': 11, 'В': 12, 'Г': 13, 'Д': 14, 'Е': 15, 'Ж': 16, 'З': 17,
    'И': 18, 'Й': 19, 'К': 20, 'Л': 21, 'М': 22, 'Н': 23, 'О': 24, 'П': 25,
    'Р': 26, 'С': 27, 'Т': 28, 'У': 29, 'Ф': 30, 'Х': 31, 'Ц': 32, 'Ч': 33,
    'Ш': 34, 'Щ': 35, 'Ъ': 36, 'Ы': 37, 'Ь': 38, 'Э': 39, 'Ю': 40, 'Я': 41,
}
SPACE_CODE = 99

def extended_gcd(a, b):
    if a == 0:
        return b, 0, 1
    gcd, x1, y1 = extended_gcd(b % a, a)
    x = y1 - (b // a) * x1
    y = x1
    return gcd, x, y

def mod_inverse(e, phi):
    gcd, x, y = extended_gcd(e, phi)
    if gcd != 1:
        raise ValueError("Обратный элемент не существует")
    return x % phi

def bin_pow(a, k, n):
    b = 1
    a = a % n
    while k > 0:
        if k % 2 == 0:
            k //= 2
            a = (a * a) % n
        else:
            k -= 1
            b = (b * a) % n
    return b

def text_to_numbers(text):
    nums = []
    for char in text.upper():
        if char == ' ':
            nums.append(SPACE_CODE)
        elif char in RUSSIAN_ALPHABET:
            nums.append(RUSSIAN_ALPHABET[char])
    return nums

def numbers_to_text(nums):
    reverse_map = {v: k for k, v in RUSSIAN_ALPHABET.items()}
    reverse_map[SPACE_CODE] = ' '
    return ''.join(reverse_map.get(num, '?') for num in nums)

def split_into_blocks(nums, n):
    blocks = []
    current = 0

    for num in nums:
        candidate = current * 100 + num
```

```

        if candidate < n:
            current = candidate
        else:
            if current > 0:
                blocks.append(current)
            current = num

    if current > 0:
        blocks.append(current)

    return blocks

def blocks_to_numbers(blocks):
    nums = []
    for block in blocks:
        temp = block
        block_nums = []
        while temp > 0:
            block_nums.insert(0, temp % 100)
            temp //= 100
        nums.extend(block_nums)
    return nums

def generate_multiple_key_pairs(p, q, count=3):
    if count < 3:
        count = 3

    n = p * q
    phi = (p - 1) * (q - 1)

    key_pairs = []
    used_e_values = set()

    print(f"\n[Генерация {count} пар ключей]")
    print(f"n = {p} * {q} = {n}")
    print(f" $\phi(n) = (p-1)(q-1) = \phi$ ")

    for i in range(count):
        attempts = 0

        while attempts < 1000:
            attempts += 1

            e = random.randint(3, phi - 1)
            gcd_result, _, _ = extended_gcd(e, phi)
            if gcd_result == 1 and e not in used_e_values:
                try:
                    d = mod_inverse(e, phi)

                    key_pair = {
                        'p': p, 'q': q, 'n': n, 'phi': phi,
                        'public': (e, n),
                        'private': (d, n),
                        'e': e, 'd': d
                    }

                    key_pairs.append(key_pair)
                    used_e_values.add(e)

                except:
                    pass

            print(f"\n Пара ключей #{i+1}:")
            print(f"    e = {e}")
            print(f"    d = {d}")

```

```

        print(f"    Проверка:  $e \cdot d \bmod \phi(n) = \{(e * d) \% \phi\}$ ")

        break

    except ValueError:
        continue

    if attempts >= 1000:
        print(f" Не удалось найти подходящее e для пары #{i+1}")
        break

    if len(key_pairs) < count:
        print(f"\n Сгенерировано только {len(key_pairs)} пар из {count}")

    return key_pairs, n, phi

def generate_custom_key_pair(p, q, n, phi):
    """Генерация ключевой пары с пользовательским e"""
    print(f"\n[Генерация собственного ключа]")
    print(f"Параметры: p={p}, q={q}, n={n},  $\phi(n)=\{\phi\}$ ")
    print(f"Условие:  $1 < e < \{\phi\}$ , НОД(e,  $\phi(n)$ ) = 1")

    while True:
        try:
            e = int(input(f"Введите ваше значение e ( $1 < e < \{\phi\}$ ): "))

            if e <= 1 or e >= phi:
                print(f"X Ошибка: e должно быть в диапазоне (1,  $\{\phi\}$ )")
                continue

            gcd_result, _, _ = extended_gcd(e, phi)

            if gcd_result != 1:
                print(f"X Ошибка: НОД( $\{e\}$ ,  $\{\phi\}$ ) =  $\{\text{math.gcd}(e, \phi)\} \neq 1$ ")
                continue

            try:
                d = mod_inverse(e, phi)

                key_pair = {
                    'p': p, 'q': q, 'n': n, 'phi': phi,
                    'public': (e, n),
                    'private': (d, n),
                    'e': e, 'd': d
                }

                print(f"\n✓ Собственный ключ создан:")
                print(f"    e =  $\{e\}$ ")
                print(f"    d =  $\{d\}$ ")
                print(f"    Проверка:  $e \cdot d \bmod \phi(n) = \{(e * d) \% \phi\}$ ")

                return key_pair

            except ValueError as ve:
                print(f"X Ошибка: {ve}")
                print("Попробуйте другое значение e")

        except ValueError:
            print("X Ошибка: введите целое число")

    choice = input("Попробовать снова? (y/n): ").lower().strip()

```

```

        if choice != 'y':
            return None

def save_to_file(content, default_name="output.txt"):
    print("\n" + "="*40)
    save_choice = input("Сохранить результат в файл? (y/n): ").lower().strip()

    if save_choice == 'y':
        filename = input(f"Введите имя файла (по умолчанию: {default_name}): ").strip()

        if not filename:
            filename = default_name

        if not filename.endswith('.txt'):
            filename += '.txt'

        try:
            with open(filename, 'w', encoding='utf-8') as f:
                if isinstance(content, list):
                    f.write(', '.join(map(str, content)))
                else:
                    f.write(content)

            print(f"✓ Результат сохранен в файл: {filename}")
            return True
        except Exception as e:
            print(f"X Ошибка при сохранении файла: {e}")
            return False
    return False

def get_text_input():
    print("\n" + "="*40)
    print("ИСТОЧНИК ТЕКСТА:")
    print("1. Ввод с консоли")
    print("2. Чтение из файла")

    choice = input("\nВаш выбор (1-2): ").strip()

    if choice == '1':
        print("\n" + "="*40)
        text = input("Введите текст для обработки: ").strip()
        if not text:
            print("X Текст не может быть пустым!")
            return None
        return text

    elif choice == '2':
        return read_from_file("текст")

    else:
        print("X Неверный выбор!")
        return None

def get_encrypted_blocks_input():
    print("\n" + "="*40)
    print("ИСТОЧНИК ЗАШИФРОВАННЫХ ДАННЫХ:")
    print("1. Ввод с консоли")
    print("2. Чтение из файла")

    choice = input("\nВаш выбор (1-2): ").strip()

```

```

        if choice == '1':
            print("\n" + "="*40)
            input_str = input("Введите блоки для расшифрования (через запятую):
").strip()
            if not input_str:
                print("X Данные не могут быть пустыми!")
                return None

            try:
                blocks = [int(x.strip()) for x in input_str.split(',')]
                return blocks
            except ValueError:
                print("X Неверный формат данных! Используйте числа, разделенные
запятыми.")
                return None

        elif choice == '2':
            return read_encrypted_blocks_from_file()

        else:
            print("X Неверный выбор!")
            return None

def read_from_file(data_type="текст"):
    try:
        root = tk.Tk()
        root.withdraw()
        root.attributes('-topmost', True)

        file_path = filedialog.askopenfilename(
            title=f"Выберите файл с {data_type}ом",
            filetypes=[("Текстовые файлы", "*.txt"), ("Все файлы", "*.*")]
        )

        root.destroy()

        if not file_path:
            print("Выбор файла отменен.")
            return None

        print(f"✓ Выбран файл: {file_path}")

        with open(file_path, 'r', encoding='utf-8') as f:
            content = f.read().strip()
            print(f"✓ Файл успешно прочитан. Размер: {len(content)} символов")
            return content

    except Exception as e:
        print(f"X Ошибка при чтении файла: {e}")
        return None

def read_encrypted_blocks_from_file():
    try:
        root = tk.Tk()
        root.withdraw()
        root.attributes('-topmost', True)

        file_path = filedialog.askopenfilename(
            title="Выберите файл с зашифрованными блоками",
            filetypes=[("Текстовые файлы", "*.txt"), ("Все файлы", "*.*")]
        )

```

```

root.destroy()

if not file_path:
    print("Выбор файла отменен.")
    return None

print(f"✓ Выбран файл: {file_path}")

with open(file_path, 'r', encoding='utf-8') as f:
    content = f.read().strip()

    try:
        blocks = [int(x.strip()) for x in content.split(',')]
        print(f"✓ Файл успешно прочитан. Найдено {len(blocks)}
блоков")

        return blocks
    except ValueError:
        print("X Неверный формат файла! Ожидаются числа, разделенные
запятыми.")

        print(f" Пример правильного формата: 12345, 67890, 54321")
        return None

except Exception as e:
    print(f"X Ошибка при чтении файла: {e}")
    return None

def print_menu():
    print("\n" + "="*50)
    print("ЛАБОРАТОРНАЯ РАБОТА №4: RSA")
    print("="*50)
    print("1. Генерация нескольких пар ключей")
    print("2. Шифрование текста")
    print("3. Расшифрование текста")
    print("4. Показать все сгенерированные ключи")
    print("5. Выход")
    print("="*50)

def main():
    print("="*50)
    print("ЛАБОРАТОРНАЯ РАБОТА №4: RSA")
    print("="*50)
    print("Задание: Генерация не менее трех пар открытого/закрытого ключа")
    print("="*50)

    all_key_pairs = []

    while True:
        print_menu()
        choice = input("Выберите действие (1-5): ").strip()

        if choice == '1':
            try:
                p = int(input("Введите p (простое число): "))
                q = int(input("Введите q (простое число): "))

                if p == q:
                    print("X Ошибка: p и q должны быть разными!")
                    continue

```

```

count = int(input("Сколько пар ключей сгенерировать? (минимум
3): "))

if count < 3:
    count = 3
    print(f"✓ Будет сгенерировано минимум 3 пары ключей")

print(f"\n[Генерация {count} пар ключей для p={p}, q={q}]")

new_key_pairs, n, phi = generate_multiple_key_pairs(p, q,
count)

all_key_pairs.extend(new_key_pairs)

print(f"\n✓ Успешно сгенерировано {len(new_key_pairs)} пар
ключей")

print(f"✓ Всего пар ключей в системе: {len(all_key_pairs)}")

if new_key_pairs:
    print("\n" + "="*40)
    custom_key_choice = input("Сгенерировать собственный ключ?
(y/n): ").lower().strip()

    if custom_key_choice == 'y':
        custom_key = generate_custom_key_pair(p, q, n, phi)
        if custom_key:
            all_key_pairs.append(custom_key)
            print(f"\n✓ Собственный ключ добавлен к списку")
            print(f"✓ Теперь всего пар ключей:
{len(all_key_pairs)}")

    if new_key_pairs or (custom_key_choice == 'y' and 'custom_key'
in locals()):
        print("\n" + "="*40)
        save_keys = input("Сохранить ключи в файл? (y/n):
").lower().strip()

        if save_keys == 'y':
            filename = input("Введите имя файла (по умолчанию:
keys.txt): ").strip()

            if not filename:
                filename = "keys.txt"

            if not filename.endswith('.txt'):
                filename += '.txt'

            try:
                with open(filename, 'w', encoding='utf-8') as f:
                    f.write("СГЕНЕРИРОВАННЫЕ КЛЮЧИ RSA\n")
                    f.write("="*50 + "\n\n")
                    f.write(f"Параметры: p={p}, q={q}\n")
                    f.write(f"Модуль: n={p*q}\n")
                    f.write(f" $\phi(n) = (p-1)*(q-1)$ \n")
                    f.write("="*50 + "\n\n")

                    for i, kp in enumerate(new_key_pairs, 1):
                        f.write(f"ПАРА КЛЮЧЕЙ #{i}

(случайная):\n")

                        f.write(f"  Открытый ключ (e, n):

({kp['e']}, {kp['n']})\n")

                        f.write(f"  Закрытый ключ (d, n):

({kp['d']}, {kp['n']})\n")

                        f.write(f"  Проверка:  $e*d \bmod \phi(n) =$ 

({kp['e'] * kp['d']} % kp['phi'])\n")

```



```

f.write("-"*40 + "\n\n")

if custom_key_choice == 'y' and 'custom_key'
in locals() and custom_key:
f.write(f"ПАРА КЛЮЧЕЙ #{len(new_key_pairs)
+ 1} (собственная):\n")
f.write(f"    Открытый ключ (e, n):
({custom_key['e']}, {custom_key['n']})\n")
f.write(f"    Замкнутый ключ (d, n):
({custom_key['d']}, {custom_key['n']})\n")
f.write(f"    Проверка: e*d mod φ(n) =
{((custom_key['e'] * custom_key['d']) % custom_key['phi'])}\n")
f.write("-"*40 + "\n\n")

print(f"✓ Ключи сохранены в файл: {filename}")
except Exception as e:
print(f"X Ошибка при сохранении ключей: {e}")

except ValueError as e:
print(f"X Ошибка: {e}")
except Exception as e:
print(f"X Ошибка при генерации ключей: {e}")

elif choice == '2':
if not all_key_pairs:
print("X Ошибка: сначала сгенерируйте ключи!")
continue

print("\n[ВЫБОР КЛЮЧА ДЛЯ ШИФРОВАНИЯ]")
for i, kp in enumerate(all_key_pairs):
e, n = kp['public']
is_custom = " (собственный)" if i >= len(all_key_pairs) - 1
and hasattr(kp, 'is_custom') else ""
print(f"{i+1:2d}. Открытый ключ: e={e:5d}, n={n:8d}
{is_custom}")

try:
key_idx = int(input(f"Выберите ключ (1-{len(all_key_pairs)}):
")) - 1

if key_idx < 0 or key_idx >= len(all_key_pairs):
print("X Неверный номер ключа!")
continue

key_pair = all_key_pairs[key_idx]
e, n = key_pair['public']

text = get_text_input()
if text is None:
continue

print(f"\n[ОБРАБОТКА ТЕКСТА]")
print(f"Текст: {text[:50]}..." if len(text) > 50 else f"Текст:
{text}")

nums = text_to_numbers(text)
if not nums:
print("X Ошибка: текст не содержит русских букв!")
continue

print(f"Числовое представление: {nums[:20]}..." if len(nums) >
20 else f"Числовое представление: {nums}")

```

```

        blocks = split_into_blocks(nums, n)
        print(f"Блоки для шифрования: {blocks}")

        encrypted_blocks = [bin_pow(block, e, n) for block in blocks]

        print(f"\n[РЕЗУЛЬТАТ ШИФРОВАНИЯ]")
        print(f"Использован ключ: e={e}, n={n}")
        print(f"Зашифрованные блоки: {encrypted_blocks}")
        print(f"Строка для передачи: {' '.join(map(str,
encrypted_blocks))}")

        save_to_file(encrypted_blocks, "encrypted.txt")

    except ValueError:
        print("X Ошибка: введите номер ключа!")
    except Exception as e:
        print(f"X Ошибка при шифровании: {e}")

elif choice == '3':
    if not all_key_pairs:
        print("X Ошибка: сначала сгенерируйте ключи!")
        continue

    print("\n[ВЫБОР КЛЮЧА ДЛЯ РАСШИФРОВАНИЯ]")
    for i, kp in enumerate(all_key_pairs):
        d, n = kp['private']
        is_custom = " (собственный)" if i >= len(all_key_pairs) - 1
        and hasattr(kp, 'is_custom') else ""
        print(f"{i+1:2d}. Закрытый ключ: d={d:5d}, n={n:8d}
{is_custom}")

    try:
        key_idx = int(input(f"Выберите ключ (1-{len(all_key_pairs)}):
")) - 1

        if key_idx < 0 or key_idx >= len(all_key_pairs):
            print("X Неверный номер ключа!")
            continue

        key_pair = all_key_pairs[key_idx]
        d, n = key_pair['private']

        encrypted_blocks = get_encrypted_blocks_input()
        if encrypted_blocks is None:
            continue

        print(f"\n[РАСШИФРОВАНИЕ]")
        print(f"Получено блоков: {len(encrypted_blocks)}")
        print(f"Блоки: {encrypted_blocks}")

        decrypted_blocks = [bin_pow(block, d, n) for block in
encrypted_blocks]

        print(f"Расшифрованные блоки: {decrypted_blocks}")

        nums = blocks_to_numbers(decrypted_blocks)
        print(f"Числовая последовательность: {nums}")

        text = numbers_to_text(nums)

        print(f"\n[РЕЗУЛЬТАТ РАСШИФРОВАНИЯ]")
        print(f"Использован ключ: d={d}, n={n}")

```

```

        print(f"Текст: {text}")

        save_to_file(text, "decrypted.txt")

    except ValueError:
        print("X Ошибка: неверный формат ввода!")
    except Exception as e:
        print(f"X Ошибка при расшифровании: {e}")

elif choice == '4':
    if not all_key_pairs:
        print("X Нет сгенерированных ключей!")
        continue

    print(f"\n[ВСЕГО СГЕНЕРИРОВАНО ПАР КЛЮЧЕЙ: {len(all_key_pairs)}]")
    print("="*70)

    for i, kp in enumerate(all_key_pairs):
        is_custom = " (собственный)" if i >= len(all_key_pairs) - 1
        and hasattr(kp, 'is_custom') else ""
        print(f"\nПАРА КЛЮЧЕЙ #{i+1}{is_custom}:")
        print(f"  p = {kp['p']}, q = {kp['q']}")
        print(f"  n = {kp['n']},  $\phi(n)$  = {kp['phi']}")
        print(f"  Открытый ключ (e, n): ({kp['public'][0]},
{kp['public'][1]})")
        print(f"  Закрытый ключ (d, n): ({kp['private'][0]},
{kp['private'][1]})")
        print(f"  Проверка:  $e*d \bmod \phi(n) = ((kp['e'] * kp['d']) \%
kp['phi'])$ ")
        print("-"*50)

    elif choice == '5':
        print("Выход из программы.")
        break

    else:
        print("X Неверный выбор! Попробуйте снова.")

if __name__ == "__main__":
    main()

```